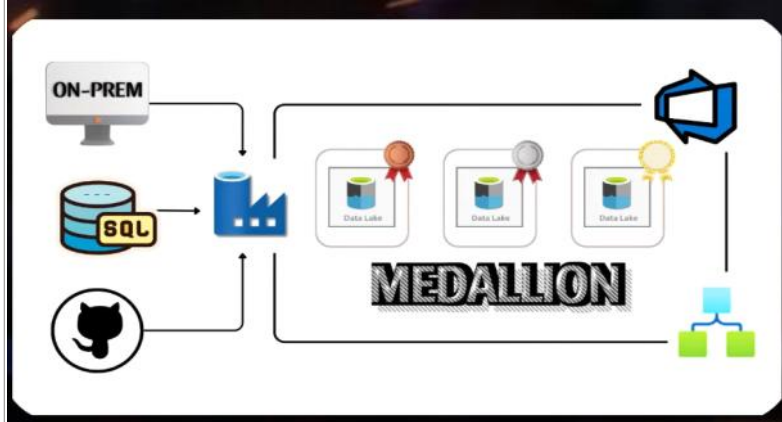


Architecture Flow Diagram :



Azure Data Factory Project II :

Start Time	Topic / Module	
0:00	Introduction	2:21:02 Incremental Data Loading
07:47	Data Architecture	2:59:05 Data Orchestration
14:23	Azure Free Account Setup	3:11:58 Logic Apps Integration
17:40	Azure Resources Overview	3:45:00 PySpark Transformation
24:24	Azure Data Factory (ADF) Tutorial	4:40:38 Gold Layer For Business Views
38:03	On-Prem To Azure Migration	5:10:20 Triggers in ADF
1:25:20	Dynamic ADF Pipelines & Mapping	5:13:51 Azure Git Integration (GitHub)
1:56:53	API To Azure Migration	5:18:59 Azure DevOps Free Account
2:12:21	Set up Azure SQL DB	5:21:21 Azure DevOps Tutorial
		5:23:31 Feature Branch & Pull Request In ADF

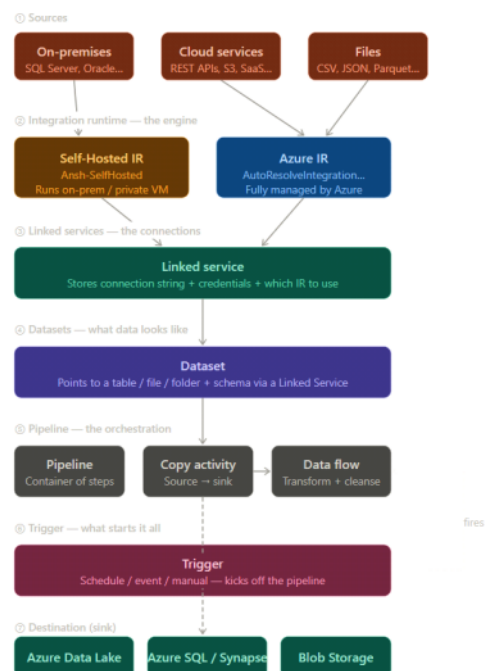
1. On Prem File System Migration to Azure

For our first pipeline, the typical flow will be like below :

Here's how all 7 layers connect in our ADF project:

- Sources** — our raw data lives here. On-premises databases, cloud APIs, or flat files. These are what we're migrating from.
- Integration Runtimes** — this is what we set up first, and now you know why. The IR is the actual compute that reaches out and reads from those sources. *XYZ-SelfHosted* handles anything behind a firewall; *AutoResolveIntegrationRuntime* handles everything cloud-facing.
- Linked Services** — think of these as saved connection strings. Each one says "connect to *this* database, using *these* credentials, via *this* IR." You'll create one per source system.
- Datasets** — once you have a connection, a Dataset says "specifically, I want *this* table / *this* folder / *this* file path." It points to a Linked Service and describes the shape of the data.
- Pipeline + Activities** — the actual workflow. A Copy Activity reads from a source Dataset and writes to a destination Dataset. A Data Flow Activity can transform/cleanse data in between.
- Trigger** — what starts the pipeline. Could be a schedule ("run every night at 2 AM"), an event ("run when a file lands in Blob"), or manual.
- Destination (sink)** — where your migrated data lands in Azure (Data Lake, Synapse, SQL DB, etc.).

Why your mentor started with IRs: Everything else depends on them. You can't create a Linked Service without choosing an IR. So the IR setup is always step zero. Click any box in the diagram to go deeper on that component!



Integration Runtime Types:

1. AutoResolveIntegrationRuntime (The "Cloud Bridge")

- Type:** Azure
- What it does:** This is the default. It lives entirely inside the Azure cloud.
- When to use it:** Use this when both your **Source** and your **Destination** are already on the internet or inside Azure (like moving data from one Azure storage folder to another).

2. XYZ-SelfHosted (The "Private Bridge")

- Type:** Self-Hosted
- What it does:** This is a piece of software installed on a specific computer or server (likely your machine or a company server).
- When to use it:** This is the key for your migration project. You use this when your **Source data** is "locked" behind a firewall (like an on-premise SQL database or a private company folder) that Azure can't see from the outside.

Why do we need to set this up first?

In a migration project, we are often moving data from an old "on-premise" system to the "cloud."

- Security:** Azure cannot just "reach into" a private company server—that would be a security risk.

- The Agent:** The **Self-Hosted IR** acts as a secure agent. It sits *inside* the private network, reaches *out* to ADF to ask "Do you have any jobs for me?", and then pushes the data up to Azure.
- Connectivity:** Without that "XYZ-SelfHosted" bridge, our ADF pipeline wouldn't be able to "talk" to our local sources; it would just get a "Connection Denied" error.

Simple Analogy

Imagine we are moving houses (migrating data).

- ADF** is the moving coordinator making a list of boxes.
- AutoResolve IR** is a standard mail truck that moves things between two public post offices.
- Self-Hosted IR** is a specialized security van that has the keys to your private gated driveway to pick up the boxes from your old basement.

What to do next: When you create your **Linked Service** (the connection to your source data), you will see a dropdown that asks which "Connect via integration runtime" to use. If your data is coming from a private server, you must select **Ansh-SelfHosted**.

Integration Runtime Setup

We will need two Integration Runtime, one is for cloud to cloud and second is for migration from ON Prem

AutoResolve Integration run time already exist, we are setting up naresh-self-hosted-IR

Key 1 : IR@See74259-349e-437a-b9a8-41b7f75d3b5c@azure7-data7-factory7@ServiceEndpoint=azure7-data7-factory7.eastus.datafactory.azure.net@3xt7ldmilBplXOLT+wTBqqAqAQlp1c/515x3m+vyB2s=

Key 2 : IR@See74259-349e-437a-b9a8-41b7f75d3b5c@azure7-data7-factory7@ServiceEndpoint=azure7-data7-factory7.eastus.datafactory.azure.net@wyODThsBMPESbgYJLIKaBA0DpmXUZKWxfGGW4x+uyJQ=

We are choosing option 2 Manual setup to install IR .

Added Key 1 to register

In Process

Installed Successfully both side

Quick Recap :

AutoResolveIntegrationRuntime — Azure IR used for cloud to cloud ---AutoResolveIntegrationRuntime — Azure IR
naresh-self-hosted-IR - Connecting to on-premises databases (SQL Server, Oracle, SAP, etc).

NEXT STEP : Creating Linked Service for File System

This is where are all the files in source >>> C:\Azure_project2_files_for_Migration

To handle the error above

We are going to use this in PowerShell (Running this as administrator)

```
cd "C:\Program Files\Microsoft Integration Runtime\5.0\Shared\"
.\dmgcmd.exe -DisableLocalFolderPathValidation
```

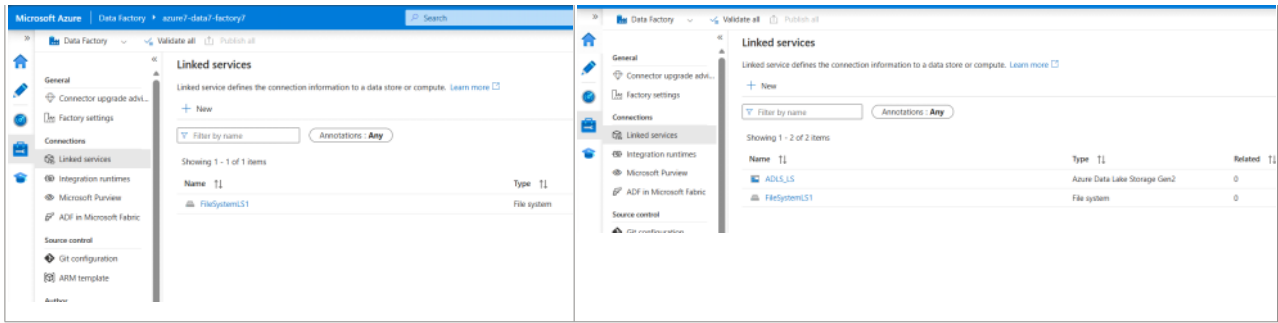
We found the above fix in stack over flow website

Successfully connected

And tested the connection , it worked and now we have Linked Service for File System

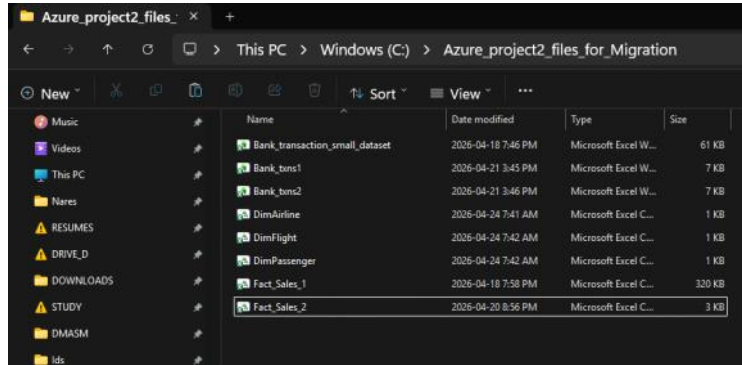
Setting Up Linked Service

We will need two Linked Service , one is for source file , one is for destination

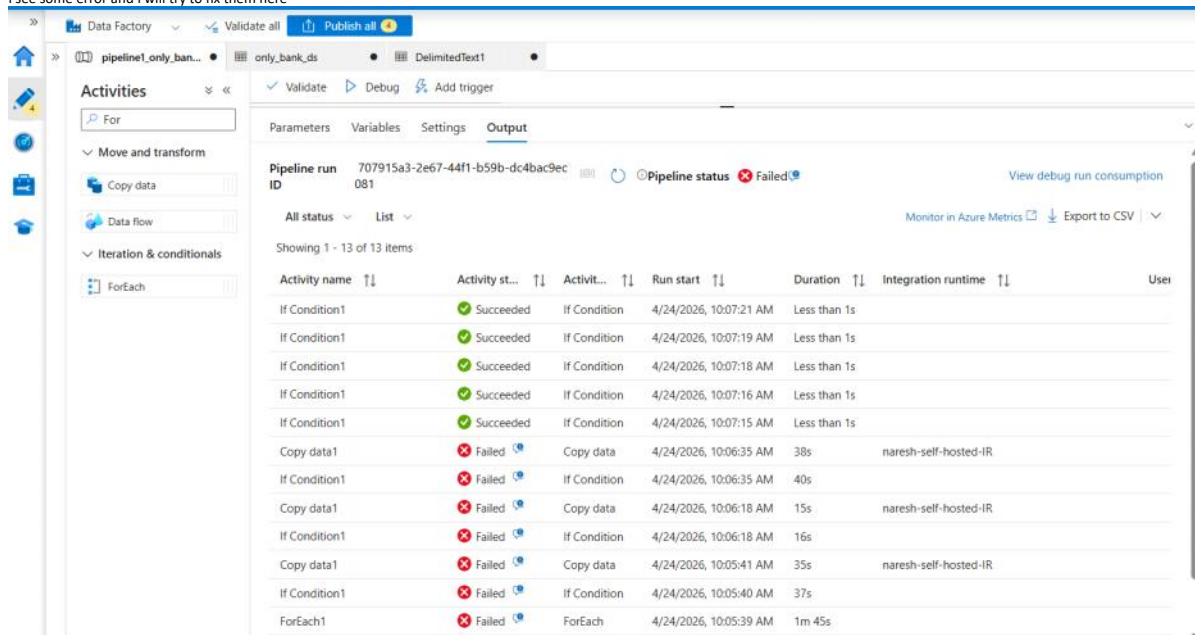


Two Linked Service has been set up one is ADLS_LS and second is FilesystemLS1

This is what our source looks like



I see some error and I will try to fix them here



Successfully debugged the pipeline

Activity name	Activity st...	Activ...	Run start	Duration	Integration runtime	User
If Condition1	Succeeded	If Condition	4/24/2026, 10:47:20 AM	Less than 1s		
If Condition1	Succeeded	If Condition	4/24/2026, 10:47:19 AM	Less than 1s		
If Condition1	Succeeded	If Condition	4/24/2026, 10:47:17 AM	Less than 1s		
If Condition1	Succeeded	If Condition	4/24/2026, 10:47:16 AM	Less than 1s		
If Condition1	Succeeded	If Condition	4/24/2026, 10:47:15 AM	Less than 1s		
Copy data1	Succeeded	Copy data	4/24/2026, 10:47:00 AM	13s	naresh-self-hosted-IR	
If Condition1	Succeeded	If Condition	4/24/2026, 10:46:59 AM	15s		
Copy data1	Succeeded	Copy data	4/24/2026, 10:46:44 AM	14s	naresh-self-hosted-IR	
If Condition1	Succeeded	If Condition	4/24/2026, 10:46:43 AM	16s		
Copy data1	Succeeded	Copy data	4/24/2026, 10:46:02 AM	39s	naresh-self-hosted-IR	
If Condition1	Succeeded	If Condition	4/24/2026, 10:46:01 AM	41s		
Copy data1	Succeeded	Copy data	4/24/2026, 10:45:24 AM	36s	naresh-self-hosted-IR	
If Condition1	Succeeded	If Condition	4/24/2026, 10:45:24 AM	38s		
Copy data1	Succeeded	Copy data	4/24/2026, 10:45:01 AM	22s	naresh-self-hosted-IR	
If Condition1	Succeeded	If Condition	4/24/2026, 10:45:00 AM	23s		
ForEach1	Succeeded	ForEach	4/24/2026, 10:45:00 AM	2m 23s		

I have successfully migrated files from on premises to azure data lake storage , In this pipeline I wanted to migrate only banking txns datasets from local host which has various kind of files

2. Data Migration Through API

Copy Data Activity vs Web Activity in ADF

Why do we need Web Activity if Copy Data exists?

Copy Data Activity is designed purely for moving/copying data from source to sink (e.g., REST API → Azure SQL, Blob → ADLS). It handles data transfer beautifully — but it has limitations:

Web Activity is a general-purpose HTTP call tool. It's used when you need to:

Use Case	Copy Data	Web Activity
Move data from API → Storage	☒ Yes	☒ No
Trigger an external REST endpoint	☒ No	☒ Yes
Get an OAuth token (e.g., Bearer token)	☒ No	☒ Yes
Call a function/webhook before pipeline	☒ No	☒ Yes
Read a small JSON response for logic	☒ No	☒ Yes
Notify Slack/Teams on success/failure	☒ No	☒ Yes

The Classic Pattern for API Migration (Your Project 2)

This is the most common real-world ADF pattern with APIs:

Web Activity (Get Token)

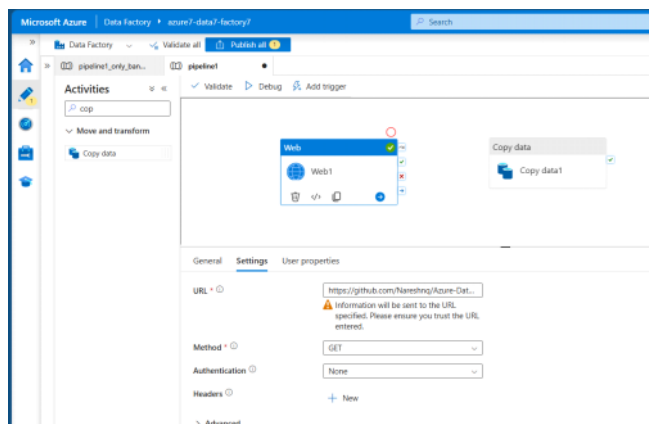
↓

Copy Data Activity (Use Token → Pull Data → Land in ADLS/SQL)

In Short

Copy Data = the truck that moves data **Web Activity** = the phone call you make before sending the truck

If your API requires authentication, pagination tokens, triggering a job first, or reading config from an endpoint — Web Activity handles that orchestration work that Copy Data simply can't do.



Pipeline2_API Migration Ran Success fully .

Using web activity we pulled the data from Git Hub using API and then use copy activity to move it to ADLS .

The screenshot shows the Microsoft Azure Data Factory console. The pipeline 'pipeline2_API Migr...' is displayed with two activities: 'Web Activity API 1' and 'Copy data1'. The pipeline status is 'Succeeded'. The 'Output' tab shows the following details:

Activity name	Activity st...	Activit...	Run start	Duration	Integration runtime	User pr
Copy data1	✓ Succeeded	Copy data	4/24/2026, 2:32:35 PM	14s	AutoResolveIntegrationRuntime (East US)	
Web Activity API 1	✓ Succeeded	Web	4/24/2026, 2:32:23 PM	11s	AutoResolveIntegrationRuntime (East US)	

3. DATA MIGRATION FROM AZURE SQL TO ADLS [Incremental Loads]

Difference between SQL Databases ,SQL managed instance and SQL Virtual Machines options

SQL Database is the most "hands-off" option. Microsoft patches it, backs it up, scales it. You just use it like any database. The trade-off is that it's a trimmed version of SQL Server — some advanced features like SQL Agent jobs, linked servers, and cross-database queries are missing. In ADF, you connect to it via the AutoResolve Azure IR with no special setup.

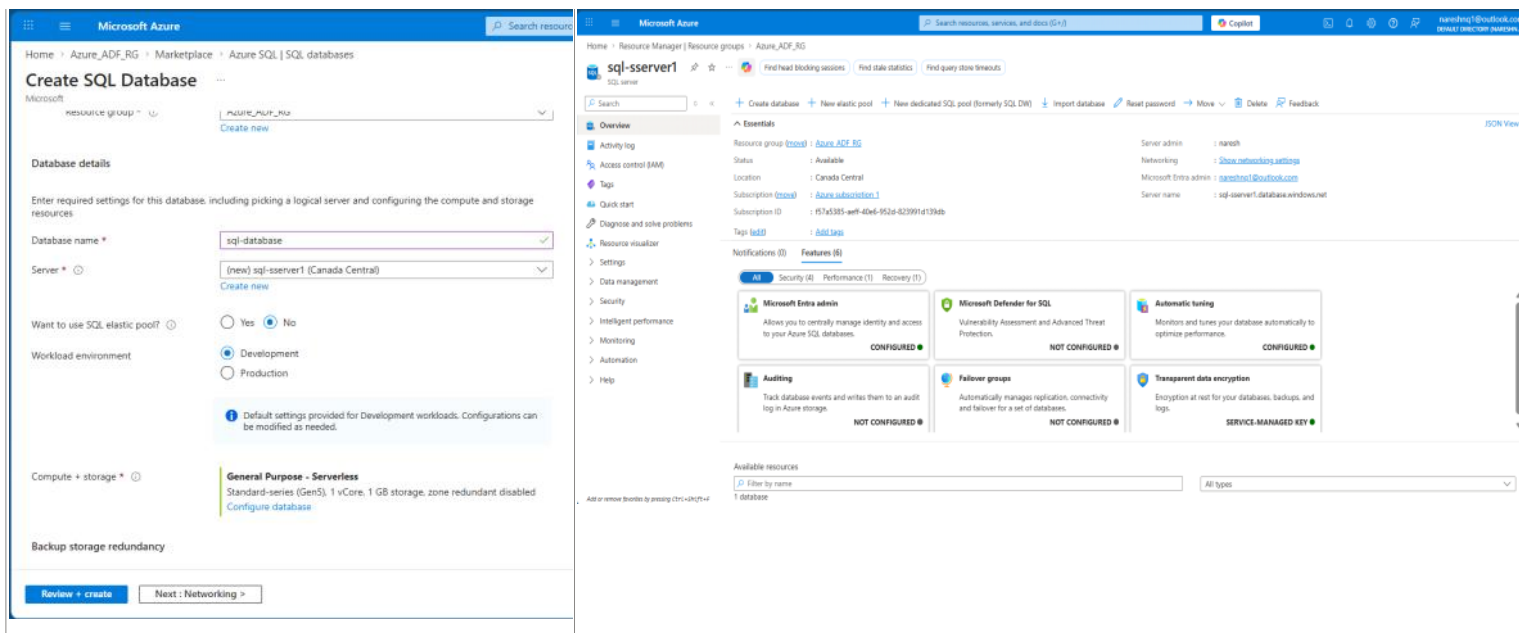
SQL Managed Instance is the sweet spot for migrations. It's still fully managed by Microsoft (no OS patching), but it gives you almost 100% SQL Server compatibility — including SQL Agent, SSIS, linked servers, and CLR. This is what we would likely recommend when migrating an existing on-premises SQL Server to Azure without rewriting anything. It connects via Azure IR if you enable a public endpoint, otherwise it needs the Self-Hosted IR.

SQL Virtual Machine is just a regular Windows VM with SQL Server installed on it. You get complete control — exact SQL version, OS configuration, third-party tools — but you're also responsible for patching, backups, and availability. Because it lives inside a virtual network, ADF almost always needs Ansh-SelfHosted (your SHIR) to reach it.

For our migration project: the source systems we're migrating *from* are likely on-premises SQL Servers (reached via SHIR), and the destination in Azure is probably SQL Database or SQL Managed Instance (reached via Azure IR). That's exactly why we've set up both IRs first.

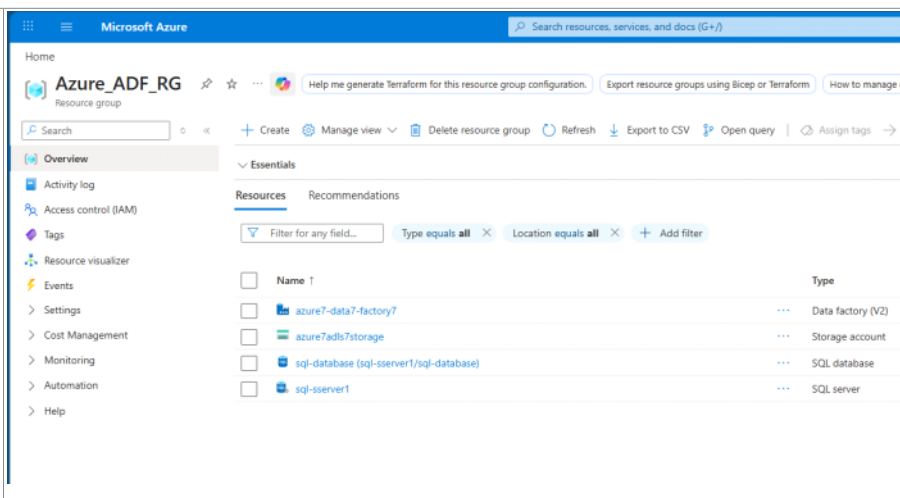
Step 1 Create two new resource Azure SQL Database and SQL server

The left screenshot shows the 'Azure SQL | SQL databases' page in the Azure portal. A search filter 'SQL database (Free offer)' is applied, resulting in 'No SQL databases to display'. The right screenshot shows the 'Create SQL Database' wizard. The 'Public endpoint' option is selected under 'Network connectivity'. The 'Firewall rules' section shows 'Allow Azure services and resources to access this server' set to 'Yes'. The 'Connection policy' section is also visible.

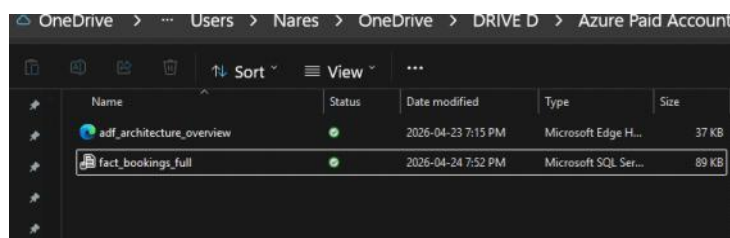


Now we have all the resources for this section of project

- Data lake
- Azure data factory
- SQL server
- SQL database



>> We will go to SQL database and create a table using 999 records . We have downloaded file name fact_bookings_full is in our local computer which helps to create table in azure sql database



Here we have updated the table FactBookings

Microsoft Azure | Search resources, services, and docs (G+/)

Home > Azure_ADF_RG > sql-database (sql-sserver1/sql-database)

sql-database (sql-sserver1/sql-database) | Query editor (preview)

SQL database

Search

New query | Templates | Open in | Feedback | Sign out | Classic experience

Explorer

- sql-database
 - dbo
 - Tables
 - FactBookings
 - Views
 - Stored Procedures
 - Functions
 - Queries
 - SQL query 1

SQL query 1

```

9 ticket_cost DECIMAL(10,2),
10 flight_duration_mins INT,
11 checkin_status VARCHAR(10)
12 );
13
14 SELECT * FROM FactBookings ;
15
16
17

```

Messages Results

	123 booking_id	123 passenger_id	123 flight_id	123 airline_id	123 origin_airpo...	123 destination...	booking_date	ticket_cost	123 flight_durati...	checkin_stat...
1	1	90	187	8	6	3	2025-04-20	167.19	178	Yes
2	2	49	110	8	9	8	2025-01-30	475.86	256	Yes
3	3	93	187	10	3	9	2025-01-23	399.53	607	Yes
4	4	18	149	3	9	2	2025-03-23	809.63	306	No
5	5	7	190	2	8	1	2025-02-01	933.30	576	No
6	6	85	191	4	8	6	2025-01-31	264.49	258	Yes
7	7	64	113	9	7	2	2025-01-20	867.46	712	No

Succeeded (0 sec 161 ms)

Columns: 10 Rows: 985

>> Lets create a new pipeline **pipeline3_sql_to_datalake**

We will use copy activity first of all and using source tab we will be selecting azure sql database as the source

Microsoft Azure | Data Factory > azure7-data7-factory7

Search

pipeline3_sql_to_d...

Activities

- Copy data
- Copy_from_SQL

General Source Sink Mapping Settings User properties

Source dataset * Select... + New

New dataset

In pipeline activities and data flows, reference a dataset to specify the location and structure of data within a data store. [Learn more](#)

Select a data store

sql

All Azure Database File Generic protocol

- Amazon RDS for SQL Server
- Azure Cosmos DB for NoSQL
- Azure Database for MySQL
- Azure Database for PostgreSQL
- Azure SQL Database
- Azure SQL Database Managed Instance

Create Linked Service along the way - **SqlDatabaseLinkedService**

And make sure you leave table name empty and click okay

New linked service

Azure SQL Database [Learn more](#)

Account selection method

From Azure subscription Enter manually

Azure subscription Select all

Server name * sql-sserver1

Database name * sql-database

Authentication type * SQL authentication

User name * Naresh

Password *

Always encrypted ☐

Encrypt ☐

Create Cancel Test connection

Set properties

Name ds_sql_source

Linked service * SqlDatabaseLinkedService

Table name Select... Enter manually

Import schema

From connection/store None

Advanced

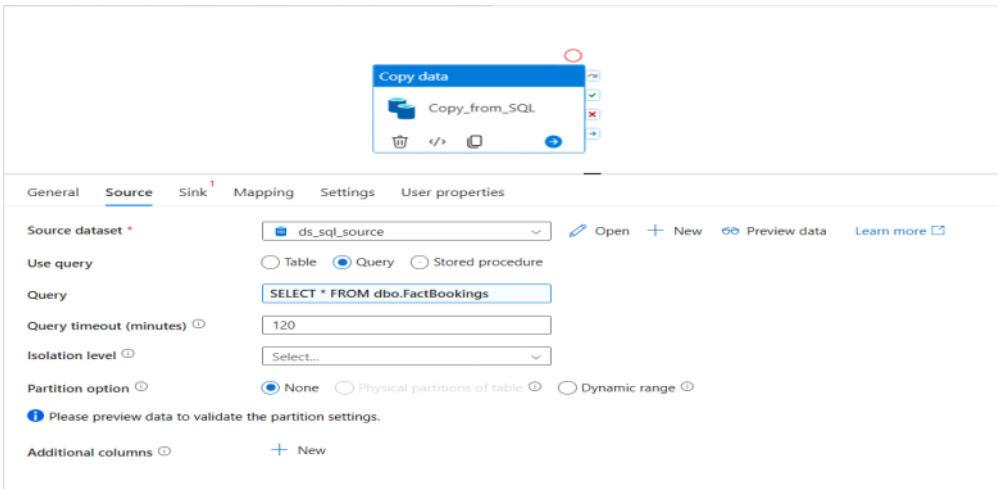
OK Back Cancel

Create Linked Service along the way - **SqlDatabaseLinkedService**

And make sure you leave table name empty and click okay

Under Copy Activity >> Source Tab >>>> Click on Query because we will be incrementally loading the data later

Below that Query* click on Add Dynamic Content >> SELECT * FROM dbo.FactBookings click ok > click preview data (We will be skipping this part as explained below)



Need of Timestamp (Watermark)

1. The Problem: Full Loads vs. Incremental Loads

When you use a simple SELECT * on a table like FactBookings, you are performing a **Full Load**.

- **The Bottleneck:** If your table has 10 million rows and you add 1,000 new bookings today, a Full Load forces Azure Data Factory (ADF) to read all 10,000,000 rows.
- **The Cost:** You waste compute power in Azure SQL, bandwidth in the Copy Activity, and storage processing time in ADLS.
- **The Risk:** As the data grows over years, the "load window" will eventually become so long that it won't finish before the next day's load starts.

2. The Solution: The Timestamp (Watermark)

To solve this, we use a specific column in your database—usually named LastUpdated, ModifiedDate, or Timestamp—to act as a **Watermark**.

Think of a Watermark like a bookmark in a book. It tells ADF: "I have already read everything up to page 500 (Yesterday's Timestamp). Today, only show me pages 501 and above."

How the Logic Works in ADF:

1. **Lookup Old Watermark:** ADF looks at a control table (or a file) to see the timestamp of the last successful load (e.g., 2026-04-23 10:00:00).
2. **Lookup New Watermark:** ADF queries the source SQL database to find the current maximum timestamp:
 - o SELECT MAX(LastUpdated) FROM dbo.FactBookings
3. **The Incremental Copy:** ADF executes a Copy Activity with a filtered query:
 - o SELECT * FROM dbo.FactBookings WHERE LastUpdated > 'OldWatermarkValue' AND LastUpdated <= 'NewWatermarkValue'
4. **Update Watermark:** Once the copy is successful, ADF saves the NewWatermarkValue so it can be used as the starting point for tomorrow's run.

3. Why This is "Modern Practice"

In modern data lake architectures (like Medallion Architecture), efficiency is king. By using timestamps:

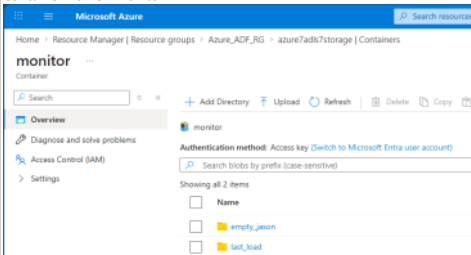
- **Speed:** Your pipeline takes seconds or minutes instead of hours.
- **Cost Efficiency:** You pay less for Data Factory "Data Integration Units" (DIUs) because the volume of data per run is small.
- **Reliability:** It is much easier to recover a failed "incremental" slice of data than it is to restart a massive 5TB full load.

Important Note on "Soft Deletes"

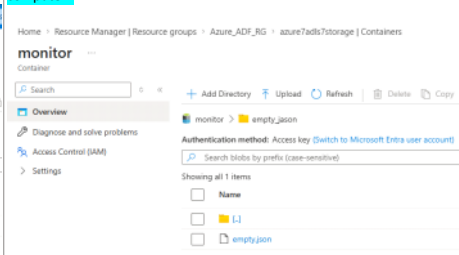
One thing the lecture might not have mentioned yet: **Timestamps are great for New and Updated rows, but they don't catch Deletes.** If a row is physically deleted from your SQL database, the timestamp doesn't change—the row just vanishes. To handle this in a truly modern practice, companies often use "Soft Deletes" (a column like isDeleted=1) so the timestamp *does* update, allowing ADF to see the change and sync it to ADLS.

>>> Creating Containers

Container Name - Monitor

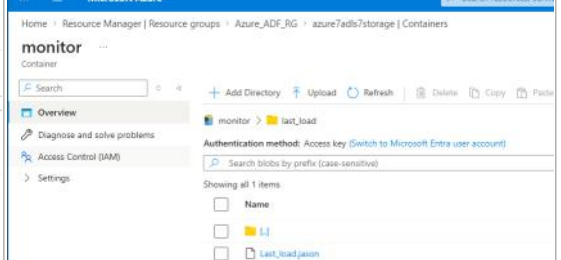


Empty_json folder will have a blank Jason file added from our local computer

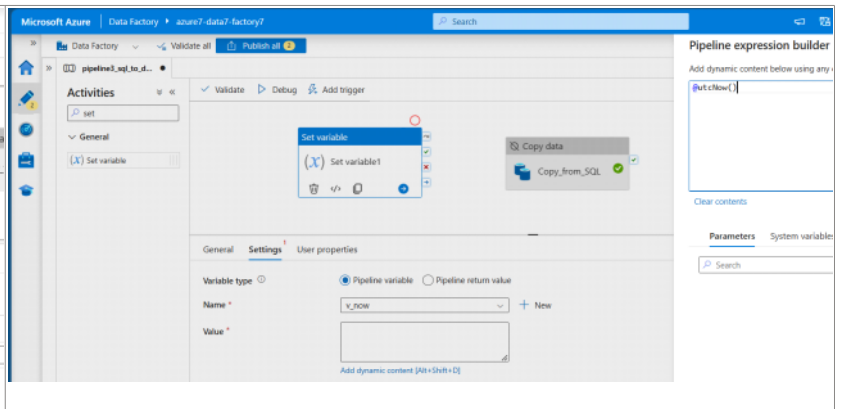
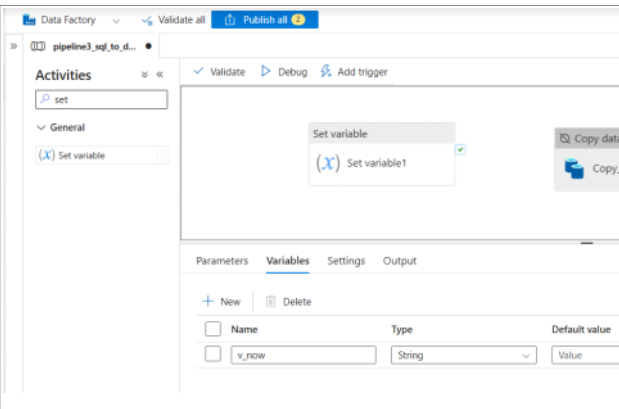


>> add another empty Jason under 'Last_Load' folder and rename it as **last_load.json** and there in this file we need to add a time stamp which we get through another activity called

Set variable activity

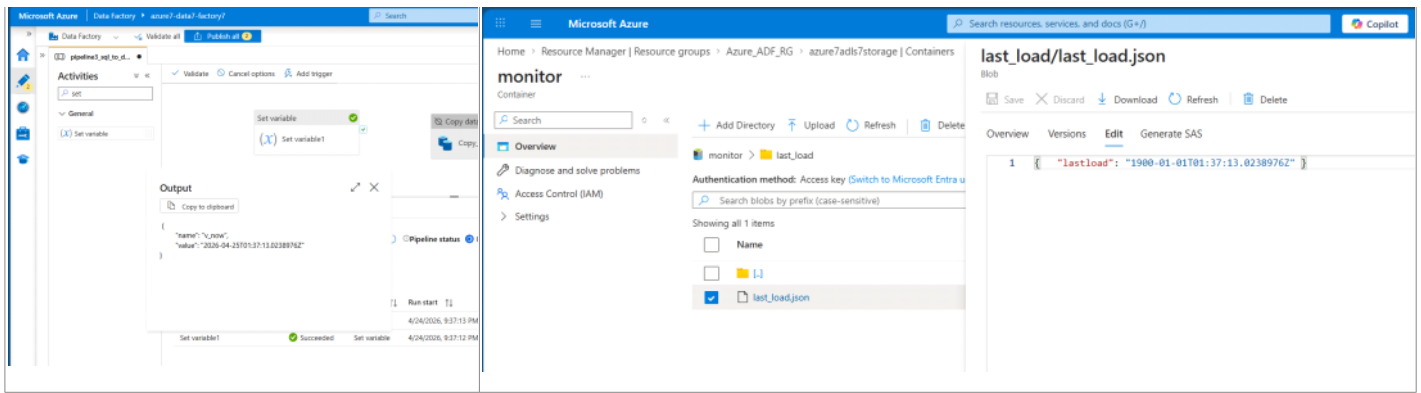


Creating set variable activity to know the current timestamp



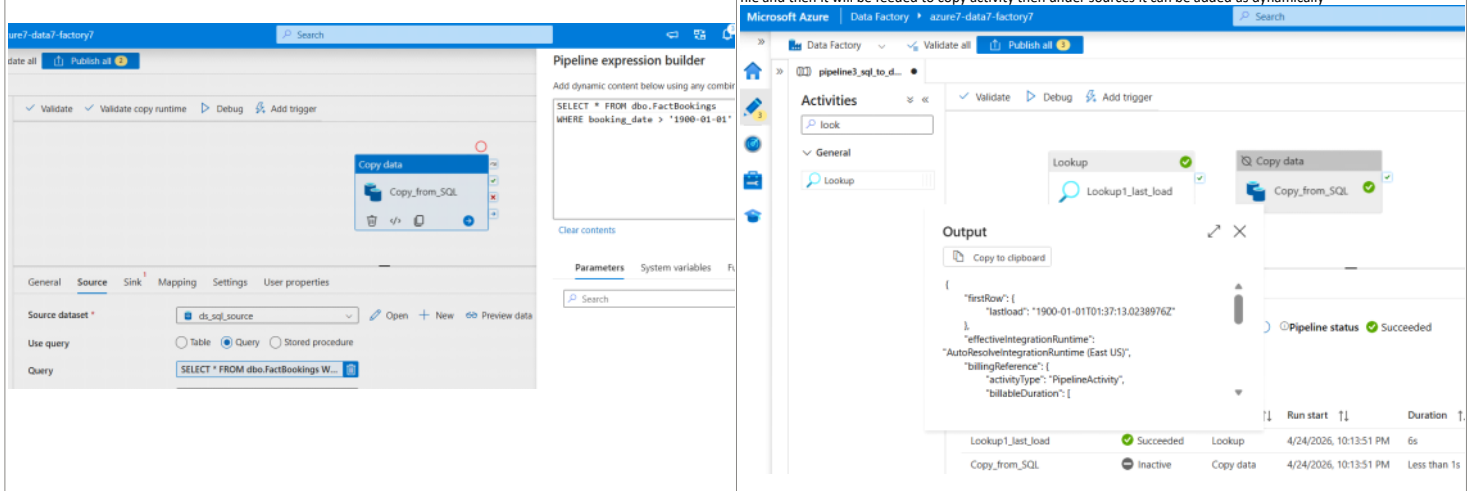
>> Now lets run the activity and we will get the current timestamp and copy it to clip board

Added the older time stamp with the same time format to last_load json file

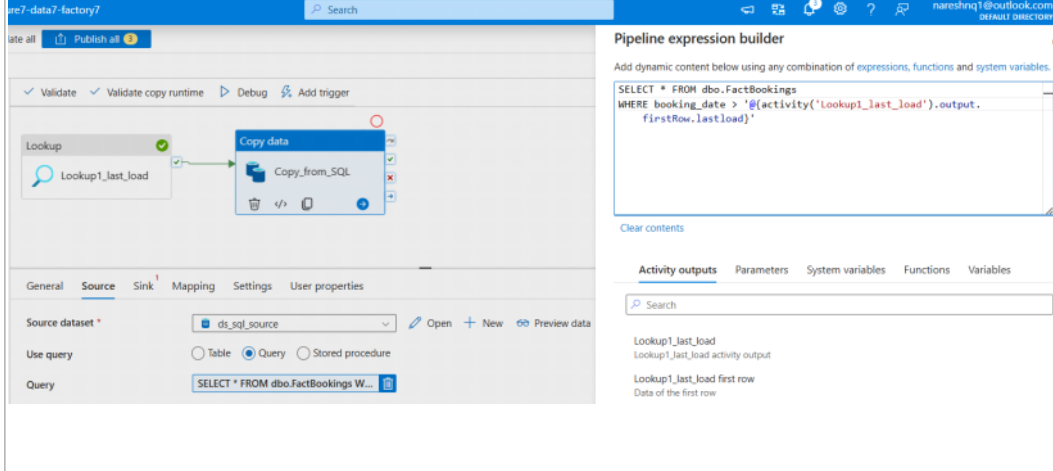


Now we can remove the set variable activity and now we can answer why did we added that time stamp to last_load.json file , Actually we want to see that go to copy data activity and go to source , we want everything where booking date is greater than 1900-01-01 (in easy language from beginning)

But it is a hardcoded value , we need dynamic value here



Now lets add this to copy activity and add dynamic query to source tab



Here is the plain English explanation of each piece:
SELECT * FROM dbo.FactBookings — standard SQL, reads all columns from the bookings table. Nothing special here.

WHERE booking_date > — this filters rows, giving us only bookings that are *newer* than a certain date.

@{...} — the single quotes around @{...} tell SQL Server to treat whatever comes out as a string/date value. The @{} is ADF's own expression language — anything inside the curly braces gets *evaluated at runtime* before the query is even sent to SQL Server.

activity('Lookup1_last_load') — this refers to the Lookup activity you placed earlier in your pipeline. Its job was to read the Last_load.json file from ADLS and return its contents.

.output — every ADF activity produces an output object after it runs. This is how you access what the Lookup activity found.

.firstRow — the Lookup activity always returns its result as firstRow (because a JSON file or a single-row query returns one row). This unwraps that row.

.lastload — this is the exact key name from your JSON file: { "lastload": "1900-01-01..." }. ADF reads that key and pulls out its value.

ERROR

Error details

Error code: DEDICATED_DPERRORCODE0

Details: Conversion failed when converting date and/or time from character string.

Activity ID: 53d29d07-790d-4bb0-b969-e3bfeb4643dd

Properties

General

Name: pipeline3_sql_to_datalake

Description:

Annotations: + New

FIXED IT - AS I was putting "firstRow": {"lastload": "1900-01-01T00:00:00Z"} while previewing the data, It was just 1900-01-01T00:00:00Z

Preview data

Linked service: SqlDatabaseLinkedService

Object:

booking_id	passenger_id	flight_id	airline_id	origin_airport_id	destination_airport_id	booking_date
1	1	90	187	8	6	3
2	2	49	110	8	9	8
3	3	93	187	10	3	9
4	4	18	149	3	9	2
5	5	7	190	2	8	1
6	6	85	191	4	8	6
7	7	64	113	9	7	2
8	8	69	144	8	7	9
9	9	93	176	8	10	7
10	10	8	108	4	5	3

Properties

General

Name: pipeline3_sql_to_datalake

Description:

Annotations: + New

Now we need another lookup activity for the latest load which will contain the maximum value of latest load

Pipeline expression builder

Add dynamic content below using any combination of expressions, functions and system variables.

SELECT MAX(booking_date) AS LatestLoad FROM dbo.FactBookings

Output

Copy to clipboard

```
{
  "firstRow": {
    "LatestLoad": "2025-06-30T00:00:00Z"
  },
  "effectiveIntegrationRuntime": "AutofocusIntegrationRuntime (Canada Central)",
  "billingReference": {
    "activityType": "PipelineActivity",
    "billableDuration": {
      "Lookup1_LatestLoad": 14s,
      "Lookup1_LastLoad": 18s
    }
  }
}
```

Pipeline status Succeeded

Activity	Status	Run start	Duration
Lookup1_LatestLoad	Succeeded	4/24/2025, 11:00:42 PM	18s
Lookup1_LastLoad	Succeeded	4/24/2025, 11:00:42 PM	14s

Now lets edit that dynamic query one more time under copy activity under source tab

Pipeline expression builder

Add dynamic content below using any combination of expressions, functions and system variables.

```
SELECT * FROM dbo.FactBookings
WHERE booking_date > '@(activity('Lookup1_last_load').output.firstRow.lastload)'
AND
booking_date <='@(activity('Lookup1_latest_load').output.firstRow)'
```

Clear contents

Activity outputs Parameters System variables Functions Variables

Search

Lookup1_last_load
Lookup1_last_load activity output

Lookup1_last_load first row
Data of the first row

Lookup1_latest_load
Lookup1_latest_load activity output

Lookup1_latest_load first row
Data of the first row

Now we can preview the data after putting the first row value from another Lookup1_latest_load which is 2025-06-30T00:00:00Z

Please provide actual value of the parameters to preview data

Please provide actual value of the parameters for pipeline pipeline3_sql_to_data lake.

Name	Type	Value
No records found		

Additional expressions

Name	Type	Value
@activity('Lookup1_last_load...')	String	1900-01-01T00:00:00Z
@activity('Lookup1_latest_load...')	String	2025-06-30T00:00:00Z

Preview data

Object

booking_id	passenger_id	flight_id	airline_id	origin_airport_id	destination_airport_id	booking_date
1	1	50	107	5	3	04/05/2023
2	2	40	110	5	9	01/05/2023
3	3	50	107	13	3	01/05/2023
4	4	10	145	3	9	02/05/2023
5	5	7	160	2	8	02/05/2023
6	6	85	109	4	8	01/05/2023
7	7	64	112	9	2	01/05/2023
8	8	50	144	8	7	06/10/2025
9	9	50	175	8	13	02/10/2025
10	10	9	188	4	5	06/10/2025

Lets complete sink tab as well and save the files in parquet format :

Set properties

Name
Parquet1

Linked service *
ADLS_LS

File path
bronze / sql / File name

Import schema
☒ From connection/store ☐ From sample file ☐ None

> Advanced

Sink dataset *
Select... + New

OK Back Cancel

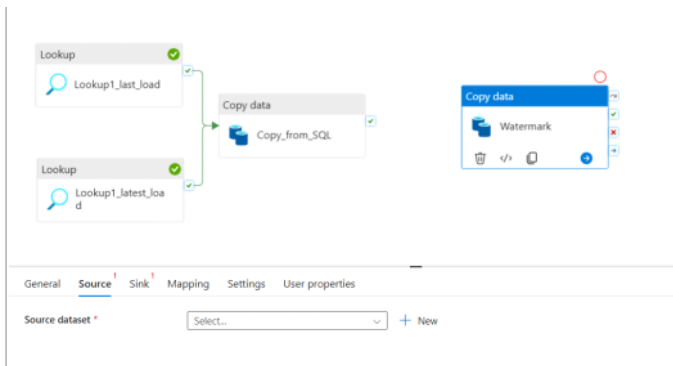
Now the copy activity will save the data in parquet format in bronze folder

Need of another copy activity , Water mark activity

Copy Activity (Watermark) >>

This is **NOT** copying business data. It's doing something completely different:
 It writes the **new watermark value** back into a watermark table so the next pipeline run knows where to start from.
 Watermark Table (before run): 2024-01-01
 Copy_from_SQL pulls: Jan 1 → Apr 25 data
 Watermark activity writes: 2025-04-25 ← updates the table
 Next run will start from: 2025-04-25 ✓

Copy_from_SQL = moves your **business data** Copy Watermark = moves the **checkpoint/bookmark** so next run is incremental
This is the backbone of incremental load pipelines in real-world ADF projects



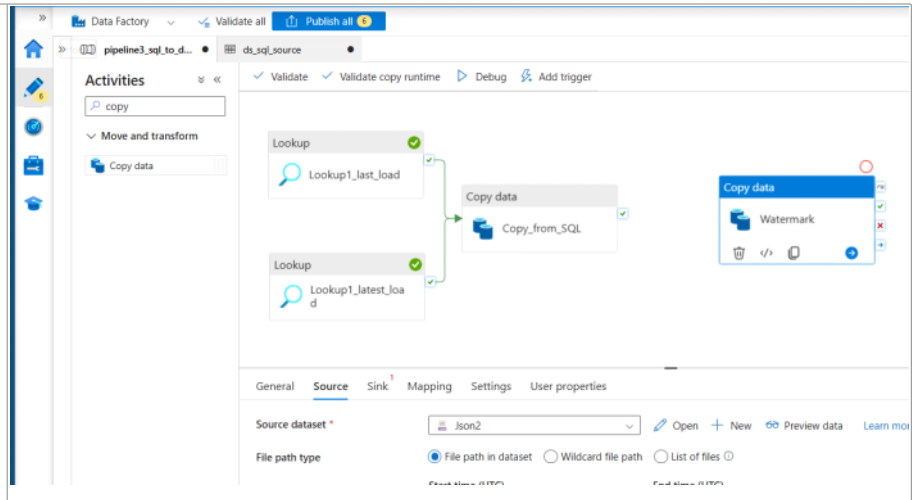
>>
 Under source tab we will choose empty data set that we have under monitor container, choose json format, choose empty json file

Question: Why do we want to choose an empty json file, what will we copy there?

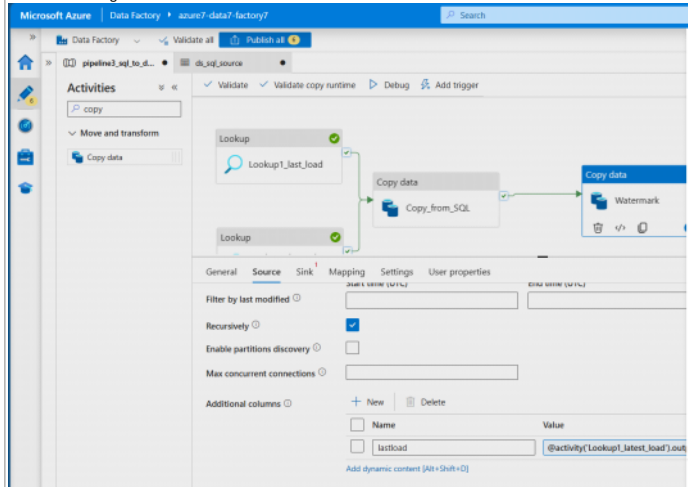
Answer --
 The Watermark Copy activity is NOT copying FROM the empty JSON. The empty JSON file is just a dummy/placeholder source — because ADF's Copy Data activity always requires a source AND a sink. It can't exist with just a sink.

Think of it like this:

ADF Copy activity = a truck Every truck needs a pickup location (source) AND a drop-off location (sink) But for the watermark update, you only care about the drop-off So you give the truck a fake/empty pickup location just to satisfy ADF's requirement



We will be adding a column to watermark table



What is "Additional Columns"?

This is how you inject a custom value into your data that doesn't exist in the source file. Remember — your source is an **empty JSON** file (no real data). So ADF lets you say:
"Even though my source has nothing, I want to ADD this extra column with this value when writing to the sink."

Because you **can't** **hardcode** the date — it changes every run. The dynamic expression automatically picks up whatever the latest timestamp is at runtime, so your watermark is always accurate and your pipeline is fully **automated and incremental**. ☺

Breaking Down What You See

Name Value

lastload → @activity('Lookup1_latest_load').output.firstRow.latestload

Name = lastload

This is the **column name** that will be created and written into your watermark table.

Value = @activity('Lookup1_latest_load').output.firstRow.latestload

This is the **dynamic expression** that fetches the latest timestamp from your second Lookup activity.

Breaking the expression down:

@activity('Lookup1_latest_load') → grab output from this lookup

.output → its output object

.firstRow → take the first (only) row returned

.latestload → get this specific column value

So if your Lookup1_latest_load ran a query like:

sql

SELECT MAX(UpdatedDate) AS latestload FROM SourceTable

Then .firstRow.latestload = 2025-04-25 10:30:00 ← that exact value gets written.

Lets test all activities now :

Before :

Activities

- Lookup1_Last_Load
- Lookup1_Latest_Load
- Copy data (Copy_From_SQL)
- Watermark

Pipeline run

Pipeline run ID: 09867b3-35c4-424f-bd8f-7650e77171

Pipeline status: Succeeded

Activity run details:

Activity name	Activity status	Activity type	Run start	Duration	Integration runtime	Used
Copy data (Copy_From_SQL)	Inactive	Copy data	4/24/2026, 11:03:56 PM	Less than 1s	Unknown	
Lookup1_Last_Load	Succeeded	Lookup	4/24/2026, 11:03:42 PM	18s	AutoResolveIntegrationRuntime (Canada Central)	
Lookup1_Last_Load	Succeeded	Lookup	4/24/2026, 11:03:42 PM	14s	AutoResolveIntegrationRuntime (East US)	

After : we can see it successfully ran all activities

Activities

- Lookup1_Last_Load
- Lookup1_Latest_Load
- Copy data (Copy data1)
- Watermark

Pipeline run

Pipeline run ID: 8808d7c-0214-411e-b42d-bb02debed4

Pipeline status: Succeeded

Activity run details:

Activity name	Activity status	Activity type	Run start	Duration	Integration runtime	Used
Watermark	Succeeded	Copy data	4/26/2026, 12:13:59 AM	15s	AutoResolveIntegrationRuntime (East US)	
Copy data1	Succeeded	Copy data	4/26/2026, 12:13:41 AM	18s	AutoResolveIntegrationRuntime (East US)	
LatestLoad	Succeeded	Lookup	4/26/2026, 12:13:30 AM	7s	AutoResolveIntegrationRuntime (Canada Central)	
LatestLoad	Succeeded	Lookup	4/26/2026, 12:13:30 AM	7s	AutoResolveIntegrationRuntime (East US)	

We can also see the date under lastload.json file date has changed from 1900-01-01 to 2025-06-30, Means it has been updated

lastload/lastload.json

```
{
  "LastLoad": "2025-06-30T00:00:00"
}
```

In Details of copy activity we see the data has shrunk to more than half , because of the parquet format . That is why we use delta lake in modern time As it compresses the data and reduce the cost .

Details

Activity run ID: d78a123-3a7d-4dee-8137-80acd387955

Data read:

- Data read: 60,054 KB
- Rows read: 985
- Peak connections: 1

Data written:

- Data written: 26,572 KB
- Files written: 1
- Rows written: 985
- Peak connections: 1

Copy duration: 00:00:18

Throughput: 7,507 KB/s

Azure SQL Database → Azure Data Lake Storage Gen2

Start time: 4/25/2026, 6:46:56 PM

Used DDU: 4

Used parallel copies: 1

Duration: 00:00:18

Details:

- Queue: 00:00:00
- Transfer: 00:00:00
- Time to first byte: 00:00:00
- Reading from source: 00:00:00
- Writing to sink: 00:00:00

Data consistency verification: Not verified

NOTE: If we re run the file and check that timestamp of lastload. Json file has been updated or not . It will not update it as we have not inserted the data. Latest load will be the same as again we have not inserted any data yet . Now lets Increase the data into SQL table and see the difference .

Microsoft Azure | Search resources, services, and docs (G+I)

Home > Azure_ADF_RG > sql-server1 > sql-database (sql-server1/sql-database)

sql-database (sql-server1/sql-database) | Query editor (preview)

Search

Overview Activity log Tags Diagnose and solve problems Query editor (preview) Mirror database in Fabric (preview) Resource visualizer Settings Data management Replicas Sync to other databases Integrations Azure Synapse Link Stream analytics (preview) Add Azure AI Search Power Platform Security Intelligent performance

Explorer

Search

sql-database

dbo

Tables

FactBookings

Columns

Indexes

Views

Stored Procedures

Functions

Queries

SQL query 1

Run Save as view

Your queries will be lost when your session ends. Use **Save as view** to keep results as a database view, or copy your query text to save externally.

```
1 INSERT INTO FactBookings VALUES (986, 12, 183, 9, 5, 9, '2025-05-03', 987.46, 194, 'No');
2 INSERT INTO FactBookings VALUES (987, 41, 151, 6, 5, 4, '2025-05-18', 798.6, 247, 'Yes');
3 INSERT INTO FactBookings VALUES (988, 88, 106, 6, 7, 4, '2025-04-02', 308.64, 219, 'No');
4 INSERT INTO FactBookings VALUES (989, 93, 122, 7, 7, 1, '2025-01-25', 541.03, 79, 'No');
5 INSERT INTO FactBookings VALUES (990, 27, 200, 6, 9, 8, '2025-02-14', 246.29, 199, 'Yes');
6 INSERT INTO FactBookings VALUES (991, 3, 105, 10, 3, 2, '2025-01-08', 593.44, 837, 'No');
7 INSERT INTO FactBookings VALUES (992, 6, 114, 2, 3, 9, '2025-03-10', 584.64, 156, 'Yes');
8 INSERT INTO FactBookings VALUES (993, 77, 105, 10, 9, 6, '2025-03-18', 968.41, 144, 'No');
9 INSERT INTO FactBookings VALUES (994, 37, 155, 4, 8, 3, '2025-03-04', 818.64, 481, 'Yes');
```

Messages Copy messages

12:26:17 PM Started executing on line 1
Affected rows: 15
Query executed successfully

Succeeded (0 sec 414 ms)

Now we have 1000 rows in table from 985 rows :

Microsoft Azure | Search resources, services, and docs (G+I)

Home > Azure_ADF_RG > sql-server1 > sql-database (sql-server1/sql-database)

sql-database (sql-server1/sql-database) | Query editor (preview)

Search

Overview Activity log Tags Diagnose and solve problems Query editor (preview) Mirror database in Fabric (preview) Resource visualizer Settings Data management Replicas Sync to other databases Integrations Azure Synapse Link Stream analytics (preview) Add Azure AI Search Power Platform Security Intelligent performance

Explorer

Search

sql-database

dbo

Tables

FactBookings

Columns

Indexes

Views

Stored Procedures

Functions

Queries

SQL query 1

Run Save as view

Your queries will be lost when your session ends. Use **Save as view** to keep results as a database view, or copy your query text to save externally.

```
1 SELECT * FROM FactBookings ;
```

Messages Results

	booking_id	passenger_id	flight_id	airline_id	origin_airpo...	destination...	booking_date	ticket_cost	flight_durati...	checkin_stat...
1	1	90	187	8	6	3	2025-04-20	167.19	178	Yes
2	2	49	110	8	9	8	2025-01-30	475.86	256	Yes
3	3	93	187	10	3	9	2025-01-23	399.53	607	Yes
4	4	18	149	3	9	2	2025-03-23	809.63	306	No
5	5	7	190	2	8	1	2025-02-01	933.30	576	No
6	6	85	191	4	8	6	2025-01-31	264.49	258	Yes
7	7	64	113	9	7	2	2025-01-20	867.46	712	No
8	8	69	144	8	7	9	2025-06-12	254.69	416	No
9	9	93	176	8	10	7	2025-03-10	876.94	709	No
10	10	8	108	4	5	3	2025-06-27	792.65	387	No

Succeeded (0 sec 171 ms)

Lets run the pipelines again and see the changes , please not there is no data beyond 30 june 2025 , means nothing should change

Microsoft Azure | Data Factory > azure7-data7-factory7

Search

Home > Azure_ADF_RG > azure7-data7-factory7

Data Factory > Validate all Publish all

pipeline3_SqlData...

Validate Cancel options Add trigger

Lookup LastLoad

Copy data Copy data1 watermark

Output

Copy to clipboard

```
{
  "firstRow": {
    "latestload": "2025-06-30T00:00:00Z"
  },
  "effectiveIntegrationRuntime": "AutoResolveIntegrationRuntime (Canada Central)",
  "billingReference": {
    "activityType": "PipelineActivity",
    "billableDuration": [

```

Pipeline status In progress

Monitor in Azure Metrics Export to CSV

Run start	Duration	Integration runtime	User prop...	Activity run ID	Log
4/26/2026, 12:31:43 PM	13s			069ecf42-261f-4e0e-8fc2-100b1bf74ff	
4/26/2026, 12:31:13 PM	30s	AutoResolveIntegrationRuntime (East US)		8b151f06-3c4c-4181-95b2-06b96ed21867	
4/26/2026, 12:30:56 PM	12s	AutoResolveIntegrationRuntime (East US)		F3d63993-04ee-4df8-ba1a-aea1646df59c	
4/26/2026, 12:30:56 PM	16s	AutoResolveIntegrationRuntime (Canada Central)		8584fea8-4d9e-417d-8ce0-56f5b09b4ef	

LatestLoad Succeeded Lookup

We need to add some rows which are after 30 june 2025 , lets add them again

Microsoft Azure | Search resources, services, and docs (G+)

Home > Azure_ADF_RG > sql-sserver1 > sql-database (sql-sserver1/sql-database)

sql-database (sql-sserver1/sql-database) | Query editor (preview)

Search

New query | Templates | Open in | Feedback | Sign out | Classic experience

Overview | Activity log | Tags | Diagnose and solve problems | Mirror database in Fabric (preview) | Resource visualizer | Settings | Data management | Replicas | Sync to other databases | Integrations | Azure Synapse Link | Stream analytics (preview) | Add Azure AI Search | Power Platform | Security | Intelligent performance

Explorer

Search

sql-database

dbo

Tables

FactBookings

Columns

Indexes

Views

Stored Procedures

Functions

Queries

SQL query 1

SQL query 1

Your queries will be lost when your session ends. Use **Save as view** to keep results as a database view, or copy your query text to save it externally.

```

34 INSERT INTO FactBookings VALUES (1035, 75, 104, 1, 2, '2025-08-02', 94.103, 75, 'No');
35 INSERT INTO FactBookings VALUES (1035, 27, 200, 6, 9, 8, '2025-08-03', 246.29, 199, 'Yes');
36 INSERT INTO FactBookings VALUES (1036, 3, 105, 10, 3, 2, '2025-08-04', 593.44, 637, 'No');
37 INSERT INTO FactBookings VALUES (1037, 6, 114, 2, 3, 8, '2025-08-05', 504.64, 156, 'Yes');
38 INSERT INTO FactBookings VALUES (1038, 77, 105, 10, 9, 6, '2025-08-06', 968.41, 144, 'No');
39 INSERT INTO FactBookings VALUES (1039, 37, 155, 4, 8, 3, '2025-08-07', 818.64, 481, 'Yes');
40 INSERT INTO FactBookings VALUES (1040, 11, 197, 5, 9, 2, '2025-08-08', 276.88, 402, 'Yes');
41 INSERT INTO FactBookings VALUES (1041, 96, 143, 1, 8, 5, '2025-08-09', 288.3, 90, 'Yes');
42 INSERT INTO FactBookings VALUES (1042, 14, 160, 9, 3, 8, '2025-08-10', 543.73, 118, 'Yes');

```

Messages | Results

	booking_id	passenger_id	flight_id	airline_id	origin_airpo...	destination...	booking_date	ticket_cost	flight_durati...	checkin_stat...
1	1	90	187	8	6	3	2025-04-20	187.19	178	Yes
2	2	49	110	8	9	8	2025-01-30	475.86	256	Yes
3	3	93	187	10	3	9	2025-01-23	399.53	607	Yes
4	4	18	149	3	9	2	2025-03-23	806.63	306	No
5	5	7	190	2	8	1	2025-02-01	933.30	576	No
6	6	85	191	4	8	6	2025-01-31	264.49	258	Yes
7	7	64	113	9	7	2	2025-01-20	867.46	712	No

Succeeded (0 sec, 126 ms)

Columns: 10 Rows: 1045

Let's Run the pipeline again : and check the latest load and lastload

Microsoft Azure | Data Factory | azure7-data7-factory7

pipeline3_Sql_Data...

Validate | Cancel options | Add trigger

Lookup: LastLoad

Lookup: LatestLoad

Copy data

Copy data1

Output

Copy to clipboard

```

{
  "firstRow": {
    "lastload": "2025-08-13T00:00:00Z"
  },
  "effectiveIntegrationRuntime": "AutoResolveIntegrationRuntime (Canada Central)",
  "billingReference": {
    "activityType": "PipelineActivity",
    "billableDuration": [
      {
        "Run start": "4/26/2026, 12:41:58 PM",
        "Duration": "3s"
      },
      {
        "Run start": "4/26/2026, 12:41:41 PM",
        "Duration": "7s"
      },
      {
        "Run start": "4/26/2026, 12:41:41 PM",
        "Duration": "16s"
      }
    ]
  }
}

```

LatestLoad Succeeded Lookup 4/26/2026, 12:41:41 PM 16s

Microsoft Azure | Data Factory | azure7-data7-factory7

pipeline3_Sql_Data...

Validate | Debug | Add trigger

Lookup: LastLoad

Lookup: LatestLoad

Copy data

Copy data1

Output

Copy to clipboard

```

{
  "firstRow": {
    "lastload": "2025-06-30T00:00:00Z"
  },
  "effectiveIntegrationRuntime": "AutoResolveIntegrationRuntime (East US)",
  "billingReference": {
    "activityType": "PipelineActivity",
    "billableDuration": [
      {
        "Run start": "4/26/2026, 12:42:17 PM",
        "Duration": "15s"
      },
      {
        "Run start": "4/26/2026, 12:41:58 PM",
        "Duration": "18s"
      },
      {
        "Run start": "4/26/2026, 12:41:41 PM",
        "Duration": "7s"
      },
      {
        "Run start": "4/26/2026, 12:41:41 PM",
        "Duration": "16s"
      }
    ]
  }
}

```

Copy data1 Succeeded Copy data 4/26/2026, 12:41:58 PM 18s

LastLoad Succeeded Lookup 4/26/2026, 12:41:41 PM 7s

LatestLoad Succeeded Lookup 4/26/2026, 12:41:41 PM 16s

SUCCESS

Microsoft Azure | Data Factory | azure7-data7-factory7

Details

Learn more on copy performance details from here.

Activity run id: 43d8e7d7-e54d-4710-bd66-942ced9ac1de

Azure SQL Database Region: Canada Central

Succeeded

Azure Data Lake Storage Gen2

Data read: 2,618 KB

Rows read: 43

Peak connections: 1

Data written: 3,373 KB

Files written: 1

Rows written: 43

Peak connections: 1

Copy duration: 00:00:15

Throughput: 436 bytes/s

Pipeline: Azure SQL Database → Azure Data Lake Storage Gen2

Start time: 4/26/2026, 12:41:59 PM

Used DPU: 4

Used parallel copies: 1

Duration: 00:00:15

Queue: 00:00:00

Working duration: 00:00:07

Total duration: 00:00:06

Time to first byte: 00:00:00

Reading from source: 00:00:00

Writing to sink: 00:00:00

Data consistency verification: Not verified

Microsoft Azure | Containers

lastload/lastload.json

monitor

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

Authentication method: Access key

Showing all 1 items

lastload.json

```

1 {
2   "lastload": "2025-08-13T00:00:00Z"
}

```

DATA ORCHESTRATION IN AZURE

Let's create production ready pipeline :, we want to aggregate all of these pipelines , we will also set up alerts if pipelines fails .

We have an activity called execute activity for aggregation

Master_Pipeline/Execute Pipeline >>> What it is — Execute Pipeline is an activity you place inside one pipeline that *calls another pipeline*. It creates a parent-child relationship. The parent orchestrates the overall flow, the child does the actual work

Microsoft Azure | Data Factory | azure7-data7-factory7

Search

nareshnq1@outlook.com
DEFAULT DIRECTORY

Validate all | Publish all

Migrate to Fabric (Preview)

Factory Resources

Filter resources by name

Pipelines 4

Parent_Pipeline

pipeline1_only_bank_tms

pipeline2_API Migration

pipeline3_Sql_Data_migration_to_AD...

Change Data Capture (preview) 0

Datasets 14

Data flows 0

Power Query 0

Parent_Pipeline

Validate | Debug | Add trigger

Execute Pipeline

Execute pipeline1...
pipeline1_only_bank...

Execute Pipeline

Execute_pipeline2...
pipeline2_API...

Execute Pipeline

Execute_pipeline3...
pipeline3_Sql_Data...

Parameters

Variables

Settings

Output

+ New

Properties

General | Related

Name *
Parent_Pipeline

Description

Annotations
+ New

Cancel